

Project explanation

Executive summary

Retail planners face a paired problem: estimate demand ahead, then decide whether inventory and lead time justify action. This project demonstrates an end-to-end decision-support path from transaction rows to forecasts and inventory recommendations.

Problem and objectives

The demo answers how much demand to expect, which forecast baseline has lower holdout error, how much safety stock and reorder point follow from declared assumptions, and which products merit review. It aims to be reproducible, transparent about uncertainty, and easy to run locally.

Architecture

CSV / synthetic source

- > input validation and cleaning
- > daily SKU demand panel
- > lag/calendar features
- > chronological model comparison
- > forecast horizon
- > stock policy calculations
- > CSV / Streamlit / FastAPI / SQL reporting

The Python package contains reusable logic. Streamlit and FastAPI call the same service, avoiding separate calculation implementations. SQL scripts describe a warehouse landing shape; database connectivity is not wired into the demo.

Data and provenance

The bundled generator is deterministic and produces 8 fictional SKUs with daily seasonality, weekly effects, promotions, stock, and supplier cost/lead-time assumptions. It is useful for software demonstrations, not for claiming observed demand, forecast accuracy, recovered sales, or savings. Public or company data may only be added when license and privacy terms permit.

Forecasting

Naive and moving-average baselines are necessary references. The tree model uses prior demand lags, shifted rolling means, and calendar fields. Models are scored on a contiguous future holdout to avoid random-split leakage. MAE and RMSE are reported in units; WAPE aggregates absolute error relative to total actual volume and is undefined for a zero-volume sample.

Inventory decision logic

For target service probability p , $z = \text{NormalQuantile}(p)$. Under an independent, stationary daily-demand approximation and fixed lead time L :

$\text{safety stock} = z * \text{daily demand standard deviation} * \sqrt{L}$

reorder point = mean daily demand * L + safety stock

EOQ = $\sqrt{2 * \text{annual demand} * \text{order cost} / \text{annual holding cost per unit}}$

When stock is below reorder point, a target-stock gap is rounded up and compared with MOQ. This is a teaching policy, not a complete inventory optimizer. It omits open purchase orders, supplier calendars, pack multiples, expiry, budget, capacity, lost-sales/backorder choice, and uncertain lead times.

How to explain the work

Start with the business question. Explain the input grain and data quality rules; show why a chronological split matters; compare baselines before the ML model; discuss WAPE's zero-demand limitation; then trace one SKU through safety stock and reorder point. End with synthetic data caveats and a production validation plan.

The repository also provides CSV ingestion via CLI and Streamlit, EDA summaries, SQL examples, reproducible notebook walkthroughs, an API, and an optional publication-assets builder that creates PDFs, an Excel dictionary, a slide deck, and SVG diagrams from the implementation.

Business value and scope

The system structures conversations around service level, supplier lead time, working capital, and uncertainty. It does not prove cost reduction or guarantee a service improvement. Those outcomes require representative historical backtests and a controlled pilot with actual purchase/stock data.